ATHENA

3D AI Desktop Assistant

Product Requirements, Technical Architecture, Security Model,
and Implementation Specification

Version 1.0 | October 2026

"Your intelligent companion, alive on your desktop."

Document Control

Field	Value
Document	Athena 3D AI Desktop Assistant - Product & Technical Specification
Version	1.0
Status	Initial product specification
Primary platform	macOS first; Windows and Linux planned
Core stack	Electron, React, TypeScript, Vite, Three.js, React Three Fiber, Tailwind CSS, Zustand
Purpose	Define the product behavior, technical architecture, security constraints, MVP scope, acceptance criteria, and implementation roadmap for Athena.

Executive Summary

Athena is a desktop AI companion represented by a small, animated 3D character that lives above normal desktop applications. Unlike a traditional chatbot confined to a rectangular window, Athena is designed to feel like an intelligent entity inhabiting the user's computer: floating around the screen, reacting to interaction, opening supported applications, conversing by text or voice, and performing explicitly approved desktop actions.

North Star

Athena should feel alive, intelligent, helpful, calm, responsive, futuristic, personal, and unobtrusive. Every product and engineering decision should reinforce the idea that Athena lives on the desktop rather than merely running inside another app.

- Build a robust desktop foundation before advanced agent capabilities.
- Keep the floating character lightweight enough to remain active throughout the workday.
- Use explicit security boundaries between AI reasoning, tool selection, and OS-level execution.
- Design platform abstractions so macOS can ship first without blocking future Windows and Linux support.
- Preserve user control over visibility, movement, voice, permissions, automation, and resource usage.

Table of Contents

- 1. Product Vision and Principles
- 2. Goals, Users, and Use Cases
- 3. Scope and Release Boundaries
- 4. Technology Stack
- 5. System Architecture
- 6. Electron Window Model
- 7. 3D Character System
- 8. Athena Behavior State Machine
- 9. Floating, Movement, and Multi-Monitor Logic
- 10. Interaction and User Experience
- 11. AI Conversation Architecture
- 12. Tool and Desktop Automation Framework
- 13. Application Navigation and Platform Services
- 14. Voice Interaction
- 15. Memory and Personalization
- 16. Settings and Control Center
- 17. Security, Privacy, and Permissions
- 18. Performance, Power, Accessibility, and Fullscreen Behavior
- 19. Persistence, Reliability, Logging, and Offline Behavior
- 20. Repository Structure and Key Interfaces
- 21. Testing Strategy
- 22. Implementation Roadmap
- 23. Acceptance Criteria
- 24. Risks and Mitigations
- 25. Future Roadmap and Expansion

1. Product Vision and Principles

The product is defined by the feeling of an intelligent companion living on the desktop, not by a chat window.

1.1 Vision

Athena is a 3D AI desktop assistant that can float across the user's screen, communicate naturally, react to interaction, and coordinate safe desktop actions. The floating character is the primary emotional surface; the control center exists for deeper conversations, settings, activity review, and advanced configuration.

1.2 Product principles

- Alive, not static: idle motion, gaze, reactions, and state-driven animation should create presence without constant distraction.
- User-controlled: the user can hide Athena, pin her, disable random movement, change size, mute voice, restrict permissions, or enable Do Not Disturb.
- Safe by construction: the AI may request a structured tool action, but the renderer never executes arbitrary shell output.
- Incremental capability: core floating behavior must be stable before AI, voice, screen understanding, or multi-step agents are added.
- Low-friction: common commands such as opening VS Code or hiding Athena should be fast and direct.
- Extensible: providers, platform integrations, character skins, voice systems, and new tools should plug into defined interfaces.

1.3 Non-goals for the initial prototype

- Unrestricted autonomous control of the computer.
- Arbitrary command execution generated by an LLM.
- Full screen-reading and vision automation.
- Complex long-term vector memory.
- Production-grade lip synchronization.
- Simultaneous full support for macOS, Windows, and Linux.

2. Goals, Users, and Use Cases

2.1 Primary goals

Goal	Outcome
Desktop presence	Athena appears as a transparent, always-on-top 3D companion without a visible rectangular window.
Natural motion	Athena idles and relocates organically while staying inside usable display bounds.
Fast interaction	Clicking Athena opens a nearby mini panel for chat or voice input.
Safe assistance	Supported desktop actions execute through validated tools and permission checks.
Personal control	Movement, size, visibility, voice, permissions, and performance are configurable.
Expandable AI core	AI providers and desktop tools are isolated behind interfaces.

2.2 Representative user journeys

- "Athena, open VS Code." Athena enters a working state, launches the approved application, then shows a short success reaction.
- User drags Athena to the lower-right corner. Athena remains there temporarily and optionally resumes random movement after a configured delay.
- User disables floating mode from the tray. The floating window disappears while the control center remains accessible.
- User clicks Athena near the right edge of the screen. The mini chat panel opens to the left so it stays visible.
- User says "make yourself smaller." Athena validates the request against the configured minimum and smoothly resizes.
- User requests a potentially destructive action. Athena shows a confirmation sheet explaining exactly what will happen before execution.

3. Scope and Release Boundaries

3.1 Initial prototype (MVP)

- Electron desktop application shell
- React + TypeScript renderer
- Three.js / React Three Fiber scene
- Transparent always-on-top floating window
- Procedural 3D Athena placeholder
- Idle hover animation
- Random movement on current monitor
- Screen boundary validation
- Manual drag
- Configurable size with minimum and maximum
- Enable/disable random movement
- Pin/unpin
- Show/hide
- System tray
- Settings window
- Persistent settings
- Click-to-open mini interaction panel

3.2 Post-MVP capabilities

Release area	Capabilities
Interaction	Context menu, richer reactions, advanced smart positioning, keyboard shortcuts
AI	Provider abstraction, streaming chat, tool calls, conversation history
Desktop tools	Open/focus apps, list running apps, open URLs, notifications
Voice	Push-to-talk, STT, TTS, visual listening/thinking/speaking states
Memory	Preferences, session memory, later semantic memory
Advanced agent	Screen understanding, multi-step planning, broader automations

4. Technology Stack

Layer	Technology	Purpose
Desktop runtime	Electron	Desktop windows, tray, IPC, OS integration

Renderer	React + TypeScript	Application UI and component architecture
Build	Vite	Fast development and bundling
3D	Three.js	Rendering engine
3D React layer	React Three Fiber	Declarative Three.js scene composition
3D helpers	@react-three/drei	Common R3F utilities
UI styling	Tailwind CSS	Design tokens and layout
App state	Zustand	Shared client state
2D animation	Framer Motion	Panels, transitions, UI micro-interactions
Validation	Zod	Runtime validation for IPC/tool inputs
Testing	Vitest + React Testing Library	Behavioral unit and UI testing

4.1 Electron security baseline

```
contextIsolation: true
nodeIntegration: false
sandbox: true
```

Only a minimal, typed preload API may cross from the renderer to privileged Electron main-process functionality.

5. System Architecture

Athena separates UI, 3D presentation, AI reasoning, policy checks, and OS-level execution so no single layer has excessive authority.

5.1 High-level architecture

```
User
  |
  v
React UI + Athena 3D Scene
  | typed preload API
  v
Electron IPC Gateway
  |
  +--> Window / Screen Manager
  +--> Settings / Persistence
  +--> AI Orchestrator --> AI Provider
  +--> Tool Router --> Permission & Confirmation Gate
                         |
                         v
                  Platform Service
                         |
                         v
                macOS Integration
```

5.2 Responsibility boundaries

Component	Owns	Must not own
Renderer	UI, R3F scene, local interaction state	Raw OS shell access
Preload	Small typed bridge	Business logic or arbitrary Node exposure
Electron main	Windows, tray, IPC, OS integration	Rendering concerns
AI orchestrator	Messages, provider calls, tool requests	Direct command execution

Tool router	Schema validation and approved actions	Unvalidated arbitrary scripts
Platform service	OS-specific implementation	AI prompt logic

5.3 Primary data flow for a desktop command

1. User asks Athena to open an app.
2. Renderer sends the conversation to the AI orchestrator.
3. AI provider returns a structured tool request such as open_application.
4. Tool router validates the payload with Zod.
5. Policy layer checks permission and whether confirmation is required.
6. Electron main invokes the platform service.
7. Platform service returns a success/failure result.
8. Athena transitions to happy or error state and reports the real result.

6. Electron Window Model

6.1 Window types

Window	Purpose	Key properties
Floating Athena	Renders only Athena and nearby mini UI	transparent, frameless, always-on-top, skip taskbar
Control Center	Full chat, activity, tasks, settings	normal resizable desktop window
Settings	Optional dedicated configuration surface	standard window or route in control center

6.2 Floating window baseline

```
new BrowserWindow({
  transparent: true,
  frame: false,
  alwaysOnTop: true,
  resizable: false,
  skipTaskbar: true
})
```

Important

The floating experience must be implemented with an actual transparent Electron window. A full-screen web page that merely simulates a floating character does not satisfy the product requirement.

6.3 Window behavior requirements

- Transparent pixels around Athena must remain visually transparent.
- Click and drag interactions must work on the character while avoiding accidental obstruction of unrelated desktop regions.
- Window position must be updated programmatically during random movement.
- The window manager must recover if the saved position is no longer visible after display topology changes.
- Show/hide must be available from the system tray and keyboard shortcut.

7. 3D Character System

7.1 Visual direction

- Tiny, cute, futuristic AI companion
- Female-inspired silhouette without aiming for photorealism
- White, black, deep navy, cyan, and neon-blue accents
- Glowing eyes and subtle energy core
- Small hover propulsion effect beneath the body
- Tasteful particles and holographic elements
- Premium sci-fi feeling without excessive gaming-style glow

7.2 Asset strategy

Development must not block on a final character asset. The initial release should create Athena procedurally from Three.js primitives. The scene and animation APIs should already support later replacement with a GLB/GLTF model and animation clips.

7.3 3D component responsibilities

Module	Responsibility
AthenaScene	Canvas, camera, lighting, performance mode, scene orchestration
AthenaModel	Model loading or procedural mesh composition
AthenaAnimationController	Maps behavior state to motion and animation clips
AthenaEffects	Particles, energy rings, hover effects, state indicators
AthenaEnvironment	Minimal lighting/environment helpers without an opaque background

7.4 Animation expectations

- Idle: slow vertical hover, subtle breathing, blinking, tiny rotation.
- Listening: movement pauses, glow increases, audio-wave effect appears.
- Thinking: orbiting particles or holographic symbols.
- Speaking: small head/body/hand gestures and pulsing energy ring.
- Working: restrained holographic loading effect.
- Happy: short positive acknowledgment animation.
- Error: subtle confused reaction; never dramatic or annoying.
- Sleeping: low-motion rest state after configurable inactivity.

8. Athena Behavior State Machine

```
type AthenaState =
  | 'idle'
  | 'floating'
  | 'listening'
  | 'thinking'
  | 'speaking'
  | 'moving'
  | 'working'
  | 'happy'
  | 'sleeping'
  | 'error'
```

8.1 Transition examples

From	Event	To
idle	Random destination selected	moving
moving	Destination reached	idle
idle	Push-to-talk starts	listening
listening	Speech captured	thinking
thinking	Tool request accepted	working
working	Tool success	happy
working	Tool failure	error
thinking	Text response begins	speaking
speaking	Response complete	idle
idle	Inactivity threshold reached	sleeping

8.2 State system rules

- State transitions must be explicit and observable.
- Visual animations should blend between states rather than snap.
- Random movement must not start while listening, dragging, speaking, or a confirmation prompt is active.
- Error state should return to idle after a brief acknowledgment unless a blocking problem remains.

9. Floating, Movement, and Multi-Monitor Logic

9.1 Random movement algorithm

9. Wait for a randomized idle period, typically 3 to 12 seconds.
10. Select a destination inside the allowed display work area.
11. Subtract Athena window size and edge padding before accepting the target.
12. Animate the Electron window from current to target coordinates over roughly 1.5 to 5 seconds depending on distance.
13. Use a smooth ease-in-out curve.
14. On arrival, return to idle and optionally perform a small personality motion.

```
safeX = clamp(targetX, display.x + edgePadding,
              display.x + display.width - athenaWidth - edgePadding)
safeY = clamp(targetY, display.y + edgePadding,
              display.y + display.height - athenaHeight - edgePadding)
```

9.2 Movement behavior constraints

- Athena must never intentionally leave the usable display area.
- Avoid constant motion; pauses are a core part of the personality.
- When the user is actively interacting, movement must pause.
- Dragging temporarily suspends random movement and stores the new position.
- After dragging, movement can resume after a configurable delay, for example 30 seconds.
- Pinned mode disables relocation but preserves idle animation.

9.3 Multi-monitor model

```
interface DisplayArea {
  id: number
  x: number
  y: number
  width: number
  height: number
}
```

Setting	Behavior
Current display	Stay on the display where Athena currently resides
Primary display	Always constrain movement to the OS primary display
Any display	Athena may choose targets on any allowed display
Selected display	Constrain Athena to a user-selected monitor

10. Interaction and User Experience

10.1 Direct manipulation

- Single click: open mini chat/interaction panel next to Athena.
- Right click: open custom futuristic context menu.
- Mouse down + drag: pause movement and move the window.
- Mouse release: clamp to bounds, persist location, start optional resume timer.
- Hover: optional brief gaze or acknowledgment reaction.

10.2 Mini panel smart positioning

Athena location	Preferred panel direction
Near right edge	Open to the left
Near left edge	Open to the right
Near top edge	Open downward
Near bottom edge	Open upward

10.3 Context menu

- Ask Athena
- Open Athena
- Listen / Stop speaking
- Stay here / Resume floating
- Move to another monitor
- Small / Medium / Large
- Settings
- Hide Athena
- Quit Athena

10.4 Design language

Token	Initial value
Primary background	#050810
Secondary	#0A1020
Glass	rgba(10, 25, 45, 0.65)
Neon blue	#00BFFF
Cyan	#00EAFF
Soft blue	#5AC8FA
Primary text	#F5FAFF
Secondary text	#8EA6C0

All visual values should be implemented as centralized design tokens rather than scattered literals.

11. AI Conversation Architecture

11.1 Conversation capabilities

- Streaming responses
- Conversation history
- Markdown and code blocks
- Copy action
- Cancel generation

- Retry failed response
- Clear conversation
- Compact mini chat plus full control-center conversation view

11.2 Provider abstraction

```
interface AIProvider {
  sendMessage(
    messages: ChatMessage[],
    tools?: AthenaTool[]
  ): AsyncIterable<AIChunk>
}
```

The product must not tightly couple to a single AI vendor. The provider layer should allow future integrations with OpenAI, Anthropic, Gemini, OpenRouter, Ollama, or another compatible service.

11.3 Athena system personality

Behavior contract

Athena is calm, intelligent, concise, warm, and proactive. She never claims an action succeeded unless the corresponding tool reports success. Sensitive, destructive, privacy-sensitive, or irreversible operations require explicit confirmation.

12. Tool and Desktop Automation Framework

12.1 Core rule

Security invariant

AI output never executes directly as an operating-system command. The model can only request registered tools with validated structured input.

```
interface AthenaTool {
  name: string
  description: string
  execute(input: unknown): Promise<ToolResult>
}
```

12.2 Initial approved tools

Tool	Purpose	Typical risk
open_application	Launch a known application	Low
focus_application	Bring a running app to foreground	Low
open_url	Open a validated URL	Low/Medium
search_web	Launch or perform an approved search	Low
get_current_time	Return current time	Low
get_running_apps	Enumerate supported running apps	Medium privacy
open_file	Open an explicitly selected/allowed file	Medium
show_notification	Display Athena notification	Low

12.3 Future tools

- create_calendar_event
- send_email
- control_music
- manage_files
- take_screenshot
- read_screen
- control_browser

12.4 Tool execution pipeline

```
AI tool request
  -> registered tool lookup
  -> Zod schema validation
  -> capability/permission check
  -> confirmation check if needed
  -> Electron main executor
  -> platform service
  -> result returned to AI + UI
```

13. Application Navigation and Platform Services

13.1 Application controller

```
interface ApplicationController {
  open(app: AppIdentifier): Promise<void>
  focus(app: AppIdentifier): Promise<void>
  isRunning(app: AppIdentifier): Promise<boolean>
}
```

13.2 Platform abstraction

```
interface PlatformService {
  openApplication(...)
  focusApplication(...)
  listApplications(...)
}
```

Implement MacOSPlatformService first. Platform-specific commands and APIs must remain outside React components so Windows and Linux implementations can be added later.

13.3 Supported natural-language commands

- "Open VS Code."
- "Open Chrome."
- "Switch to Spotify."
- "Open Finder."
- "Open Terminal."
- "Open Slack."
- "Move to my second monitor."
- "Stay here."
- "Stop floating."
- "Make yourself smaller."
- "Hide for an hour."
- "Open your settings."

14. Voice Interaction

14.1 Initial voice flow

```
Push-to-talk
  -> microphone capture
  -> speech-to-text
  -> AI processing / optional tool use
  -> response
  -> text-to-speech
  -> state returns to idle
```

14.2 Visual feedback

Voice stage	Athena behavior
Listening	Stops drifting; eyes/core glow brighter; audio-wave particles
Thinking	Orbiting particles or holographic symbols
Speaking	Subtle gesture + pulsing energy ring
Muted	No spoken output; text remains available

14.3 Scope rule

Push-to-talk should ship before an always-listening wake phrase. "Hey Athena" may be added later after privacy, reliability, and battery impact are understood.

15. Memory and Personalization

Memory layer	Examples	Initial scope
Session memory	Current conversation and tool outcomes	Yes
Preferences	Preferred IDE/browser, size, movement, voice settings	Yes
Semantic memory	Long-term facts retrieved by embeddings/vector search	Future

15.1 Memory principles

- Store only what improves the product experience.
- Keep product settings separate from conversational memory.
- Provide clear ways to inspect or reset user-controlled preferences.
- Do not introduce a vector database in the first prototype unless a concrete requirement needs it.

16. Settings and Control Center

16.1 Control center navigation

- Chat
- Tasks

- Activity
- Memory
- Settings

16.2 Settings groups

Group	Key controls
General	Launch at login, show after startup, minimize to tray, start hidden
Appearance	Size, glow, particles, animation intensity, UI scale, future skins
Floating	Enable, random movement, frequency, speed, display selection, always-on-top, fullscreen behavior
AI	Provider, provider-specific configuration, model settings
Voice	Enable, push-to-talk shortcut, TTS, voice, speed, volume
Automation	Approved desktop capabilities and future per-tool controls
Privacy	Application detection, screen awareness, screenshots, file access, microphone
Shortcuts	Open Athena, push-to-talk, show/hide
About	Version, diagnostics, links, licenses

16.3 System tray

- Open Athena
- Show Athena / Hide Athena
- Pause Floating / Resume Floating
- Mute Athena
- Settings
- Quit

17. Security, Privacy, and Permissions

17.1 Threat model summary

Threat	Mitigation
Prompt injection requests shell execution	No raw AI shell execution; registered tools only
Renderer compromise reaches Node APIs	nodeIntegration false + contextIsolation + minimal preload API
Malformed tool payload	Zod validation before execution
Destructive automation	Explicit confirmation gate
Credential leakage	Keep secrets out of renderer; use OS credential storage where possible
Sensitive logging	Redact tokens and avoid unnecessary content logging
Over-broad permissions	Conservative defaults and explicit privacy controls

17.2 Confirmation-required examples

- Delete file
- Send message or email
- Close an application where work may be unsaved
- Restart or shut down computer
- Run a terminal command

- Move or overwrite important files

17.3 Example confirmation surface

```
Athena wants to:

Delete:
~/Documents/report.pdf

[Cancel]   [Allow]
```

17.4 Secret handling

- API keys must never be bundled into client source.
- Keys should not be printed in logs.
- Prefer OS keychain/credential storage for persistent secrets.
- Renderer receives only the minimum status/configuration information required for UI.

18. Performance, Power, Accessibility, and Fullscreen Behavior

18.1 Performance requirements

- Reduce or pause expensive rendering when hidden.
- Avoid rendering at maximum frame rate when Athena is visually stationary if a lower-cost strategy is available.
- Cache models and textures.
- Dispose Three.js geometry, materials, textures, and listeners correctly.
- Throttle cursor tracking and expensive display queries.
- Debounce resize and settings updates.
- Avoid continuous polling when event-driven APIs exist.

Quality mode	Intent
Performance	Lower animation/particle cost
Balanced	Default compromise
Quality	Highest visual fidelity within safe resource bounds

18.2 Battery awareness

- Optional reduced animation on battery power.
- Optional pause of random movement below a configurable battery threshold, e.g. 20%.

18.3 Fullscreen and Do Not Disturb

- Optional hide while fullscreen applications are active.
- Do Not Disturb disables random speech and notifications and reduces animation.
- Suggested DND durations: 30 minutes, 1 hour, until disabled.

18.4 Accessibility

- Keyboard navigation
- Visible focus states
- ARIA labels
- Scalable UI
- Screen-reader-friendly control labels

- Respect prefers-reduced-motion by reducing Athena movement and effects

19. Persistence, Reliability, Logging, and Offline Behavior

19.1 Persisted state

- Athena size
- Last location
- Selected display
- Floating enabled
- Random movement
- Movement frequency
- Voice settings
- Appearance
- AI provider
- Window positions
- Shortcuts
- Permissions

Persisted data must be schema-validated before use. If display topology changes, Athena must recover to a safe visible position.

19.2 Error handling

- AI provider failure must not crash the app.
- Unavailable microphone must degrade gracefully to text.
- Failed model load should fall back to a placeholder or a clear error state.
- Failed application launch should produce a truthful failure message.
- Loss of internet should preserve local desktop features where possible.

19.3 Structured logging

Log	Examples
Allowed	Startup, window failures, tool names/results, AI request metadata, automation errors
Do not log	API keys, passwords, auth tokens, unnecessarily sensitive user content

19.4 Internal events

- ATHENA_STATE_CHANGED
- ATHENA_MOVED
- ATHENA_CLICKED
- ATHENA_LISTENING_STARTED
- ATHENA_SPEAKING_STARTED
- ATHENA_TOOL_STARTED
- ATHENA_TOOL_COMPLETED
- ATHENA_TOOL_FAILED

20. Repository Structure and Key Interfaces

```
athena/
├── electron/
│   ├── main/
```

```
│   │   ├── main.ts
│   │   ├── windowManager.ts
│   │   ├── screenManager.ts
│   │   ├── appLauncher.ts
│   │   ├── desktopActions.ts
│   │   └── ipcHandlers.ts
│   └── preload/
│       ├── preload.ts
│       └── electronApi.ts
├── src/
│   ├── app/
│   ├── features/
│   │   ├── athena/
│   │   ├── chat/
│   │   ├── voice/
│   │   ├── floating/
│   │   ├── automation/
│   │   └── settings/
│   ├── components/
│   ├── three/
│   ├── services/
│   ├── store/
│   ├── hooks/
│   ├── types/
│   ├── utils/
│   └── styles/
└── public/
    ├── models/
    ├── textures/
    └── sounds/
```

20.1 Engineering rules

- Use TypeScript wherever possible.
- Keep React components focused and avoid giant components.
- Keep OS operations in Electron main.
- Expose a typed preload bridge only.
- Keep Three.js character code separate from business logic.
- Use Zustand where shared state is truly needed; avoid needless prop drilling.
- Validate external inputs.
- Properly dispose 3D resources.
- Do not create placeholder files without a concrete purpose.
- Prefer small focused modules over speculative abstractions.

21. Testing Strategy

21.1 Test layers

Layer	Focus
Unit	Clamping, target generation, state transitions, schema validation, configuration rules
React component	Settings controls, mini panel behavior, state-dependent rendering
Electron integration	IPC contracts, tray actions, window creation, show/hide, persistence
Platform service	Known app identifiers, launch/focus outcomes, graceful failures
Manual/visual	Transparency, animation quality, drag feel, multi-display behavior, fullscreen coexistence

21.2 Priority automated tests

- Random targets always remain inside computed safe bounds.
- Minimum and maximum character size are enforced.
- Pinned mode prevents random relocation.
- Dragging suspends movement and persists the dropped position.
- Tool payloads with invalid schemas never reach executors.
- Unregistered tools are rejected.
- Sensitive tools route through confirmation.
- State transitions match expected event sequences.
- Invalid persisted display coordinates are repaired.

21.3 Testing philosophy

Prefer behavioral tests over snapshot-heavy testing. Visual quality of the floating 3D character must also be reviewed manually on target hardware because animation feel, transparent-window composition, and GPU behavior are difficult to validate with unit tests alone.

22. Implementation Roadmap

Phase	Scope	Exit condition
1 - Desktop foundation	Electron, React, TS, Vite, Tailwind, tray, secure preload/IPC	App launches securely and control center/tray operate
2 - Floating Athena	Transparent window, R3F scene, placeholder model, drag, size, movement, bounds	Athena floats reliably on desktop
3 - Interaction	Mini panel, context menu, state machine, reactions	Core interaction feels coherent
4 - AI	Provider abstraction, streaming chat, personality, structured tool requests	Athena can converse and request approved tools
5 - Desktop tools	Open/focus/list apps, open URL, safe action framework	Basic app navigation works with real results
6 - Voice	Push-to-talk, STT, TTS, voice states	End-to-end voice loop works
7 - Advanced	Memory, screen awareness, multi-step agent, skins, more platforms	Advanced capabilities can be added without redesigning foundation

22.1 Recommended MVP build sequence

15. Create Electron + Vite + React + TypeScript shell.
16. Implement typed secure preload bridge.
17. Create control-center window and tray.
18. Create dedicated transparent floating window.
19. Render a procedural Athena placeholder with R3F.
20. Implement state-driven idle animation.
21. Implement screen manager and safe bounds.
22. Implement window interpolation/random movement.
23. Add manual drag, pin, show/hide, and size settings.
24. Persist validated settings and recover from monitor changes.
25. Add mini panel and basic settings UI.
26. Add tests for movement, sizing, persistence, and IPC validation.

23. Acceptance Criteria

ID	Acceptance criterion
AC1	Starting Athena launches the Electron application successfully.
AC2	A transparent floating window appears containing the 3D Athena character.
AC3	No visible rectangular background surrounds Athena.
AC4	Athena stays above ordinary desktop windows when configured to do so.
AC5	Athena performs a natural idle hover animation.
AC6	Athena periodically moves to random valid locations when random movement is enabled.
AC7	Athena never intentionally moves beyond the usable screen area.
AC8	The user can manually drag Athena.
AC9	The user can disable random movement.
AC10	The user can pin Athena at the current location.
AC11	The user can change Athena's size.
AC12	Configured minimum and maximum sizes are respected.
AC13	The user can hide and show Athena.
AC14	Athena remains accessible through the system tray.
AC15	Settings survive application restart.
AC16	Clicking Athena displays a small interaction panel beside the character.
AC17	Architecture cleanly supports later AI, voice, and desktop automation.

23.1 Quality gate

- No uncontrolled raw shell execution exists.
- No Node APIs are directly exposed to the React renderer.
- Movement logic is deterministic under tests when random source is injected/mocked.
- No persistent position can strand Athena off-screen.
- Hiding Athena materially reduces 3D rendering work.
- The application remains usable if the network is offline.

24. Risks and Mitigations

Risk	Impact	Mitigation
Transparent-window GPU quirks	Visual artifacts or CPU/GPU spikes	Test early on target macOS hardware; provide quality modes and fallback effects
Always-on-top becomes intrusive	Poor user experience	Strong hide/pin/fullscreen/DND controls; conservative defaults
Random motion feels annoying	User disables core feature	Long idle pauses, low frequency default, pause during interaction
AI overreaches	Security/privacy risk	Structured tools, permission checks, confirmations, honest result reporting
Platform-specific app launching is brittle	Commands fail across machines	App identifiers, discovery layer, platform abstraction, graceful errors
Final 3D model delays development	Project stalls	Procedural placeholder and asset-independent scene APIs
Resource consumption too high	Battery drain	Performance modes, hidden-state

		suspension, throttling, disposal audits
Multi-monitor coordinate complexity	Off-screen positioning	Central screen manager, work-area normalization, tests for negative coordinates

25. Future Roadmap and Expansion

25.1 Screen awareness

```
Screen Capture
  -> Vision Model
  -> UI Understanding
  -> Action Planning
  -> Desktop Automation
```

This capability should remain opt-in and permission-gated. The first architecture should define interfaces but not build unrestricted screen automation.

25.2 Multi-step agent

```
User Request
  -> Planner
  -> Tasks
  -> Tool Calls
  -> Observations
  -> Next Action
  -> Result
```

25.3 Character evolution

- Replace procedural character with GLB/GLTF model.
- Add animation clips and higher-quality state blending.
- Support Athena Classic, Hologram, Cyber, and Minimal skins.
- Optional lip sync and better gaze/gesture control.
- Themeable particles, hover effects, and energy core.

25.4 Platform expansion

- Windows platform service
- Linux platform service
- Platform-specific packaging and auto-update strategy
- OS-specific accessibility and automation permissions

25.5 Ecosystem direction

- Plugin/tool registry with explicit permissions
- Calendar and email integrations
- Browser control
- Music controls
- File workflows
- User-authored safe automations
- Local-model mode for privacy-sensitive workflows

Appendix A - Product Behavior Reference

A.1 Default sizing

Setting	Recommended initial value
Minimum	80 px
Default	180 px
Maximum	400 px
Edge padding	16 px
Idle wait	3-12 seconds
Movement duration	1.5-5 seconds
Post-drag resume delay	30 seconds

A.2 Suggested shortcuts

Action	Default
Open Athena	Cmd/Ctrl + Shift + Space
Push to Talk	Cmd/Ctrl + Shift + A
Show/Hide Athena	Cmd/Ctrl + Shift + H

A.3 First-launch experience

27. Choose Athena size.
28. Choose floating behavior.
29. Select AI provider (can be skipped until AI phase).
30. Configure microphone (optional).
31. Review privacy permissions.
32. Finish and materialize Athena with a short holographic animation.

A.4 First appearance animation

```
particles appear
  -> energy ring forms
  -> Athena materializes
  -> Athena opens eyes
  -> subtle hover begins
```

Appendix B - Definition of the Athena Experience

Product statement

Athena should feel like a small intelligent entity living inside the user's computer rather than a chatbot running inside another window.

Athena should feel

- Alive
- Intelligent
- Helpful
- Calm
- Responsive
- Futuristic
- Personal
- Unobtrusive

Athena should not feel

- Annoying
- Constantly moving
- Constantly talking
- Distracting
- Overly animated
- Overly childish
- Intrusive

The product should demand attention only when there is a useful reason. Presence is created through subtle motion, context-aware reactions, fast access, and reliable actions rather than continuous animation or chatter.

ATHENA

Your intelligent companion, alive on your desktop.

End of Specification - Version 1.0